

Visualization & Search over Time Series Aggregates

Supplementary Material

This document is the supplementary material for the paper “Visualization & Search over Time Series Aggregates” and is meant to be read alongside it. References to Sections, Theorems, Definitions, and Figures of the form “Section IV-C” or “Theorem IV.1” refer to the main paper; items defined in this document are numbered and cross-referenced locally.

Contents

1	An End-to-End Example on Wind-Turbine Data	2
1.1	Setting	2
1.2	See: a visualization query	2
1.3	Search: a pattern query	3
1.4	Refine: resolving the ambiguous region	4
2	Motivation & Problem Formulation	5
2.1	Motivation	5
2.2	Setting and Notation	5
2.3	Problem Formulation	6
2.4	The slope problem	7
2.5	Similarities and differences of pattern and view queries, all served by the same cache	7
3	Bounding the OLS Slope from Bucket Summaries	9
3.1	Bounding the slope: intuition and proof sketch	9
3.2	Full proof	10
3.2.1	The problem, concretely	10
3.2.2	Setting up the bookkeeping	11
3.2.3	Step 1—the means (Lemma A)	12
3.2.4	Step 2—separating “between” and “within”	12
3.2.5	Step 3—bounding S_{tv} (Lemma B)	13
3.2.6	Step 4—bounding S_{tt} (Lemma C)	14
3.2.7	Step 5—from the two boxes to a slope interval	15
3.2.8	What the bound is, in one paragraph	16
4	Experiments	17
4.1	Comparison against an AQP baseline	17

1 An End-to-End Example on Wind-Turbine Data

For the reviewers: VASTA grew out of a completed project with wind-turbine operators who monitor many sensor variables visually while searching those same series for recurring patterns. The paper carries a compact version of the resulting example through its architecture (Figure 2) to leave room for the algorithms; this section gives the fuller version it was distilled from. The slope-interval numbers below are computed from the four cached aggregates using the formulas of §3, and the rendering follows the MinMaxCache pixel-error model VASTA adopts (§2.2, [3]), so each step can be checked against the paper.

1.1 Setting

Assume a control room of wind-turbine operators who visually monitor the data output of a turbine farm. Assume an analyst observing a turbine’s power output, here normalized to its rated capacity so values lie in $[0, 1]$; as wind speed climbs, the power ramps up its power curve, holds at rated output, and later falls off. An analyst session both renders and searches this series, each operation scoped to a time range $[t_s, t_e)$ and served from a single cache $\mathcal{C}$ of calendar-aligned buckets, each holding (min, max, count, sum). A view carries a tolerance $\rho_V \in [0, 1]$ on $\mathcal{E}(\text{view})$, the bounded fraction of pixels that may differ from the exact render; a search carries a tolerance $\rho_P \in [0, 1]$ on its estimated error $\mathcal{E}(\text{pattern})$. Both start coarse and refine only the regions that miss their tolerance, controlled by an aggregation factor α that splits each base unit into α sub-buckets; the sub-buckets roll up to their parent exactly.

The series is long and stored remotely, and this session interleaves seeing and searching over the same regions, so the cost that matters is the repeated backend fetch, and every design pays it somewhere. A visualization cache such as MinMaxCache [3] keeps only min and max—enough to render, but not to bound a slope—so it cannot answer the search at all; conversely, a shape-search engine such as ShapeSearch [4] or a MATCH_RECOGNIZE query computes each segment’s slope from the raw points yet keeps nothing, rescanning the range on every query. Caching the exact slope instead, by storing the full regression sums, spares that recomputation but only for search: those sums sit in a private per-query structure the view cannot read, at a heavier per-bucket footprint, and setting such a searcher beside a visualization cache merely keeps two copies of the same region, each re-fetching what the other has already summarized. Only VASTA serves both from one cache, so work done for one is available to the other. The rest of this section follows one turn of the see-search loop: rendering a window (§1.2), marking a shape and searching a longer range for it (§1.3), and refining the regions left ambiguous (§1.4).

1.2 See: a visualization query

The analyst first issues a view-query Q_V : a twelve-hour window $[t_s, t_e)$ rendered onto a 600×400 viewport at tolerance $\rho_V = 0.1$. Each of the 600 columns spans $720 \text{ min} / 600 = 1.2 \text{ min}$ of the window. Under the MinMaxCache model the paper adopts, a column is rendered exactly by any bucket lying fully inside it, whereas a bucket straddling a column boundary is uncertain there and counts toward $\mathcal{E}(\text{view})$; the view meets its tolerance once every column holds a fully-contained bucket, which is guaranteed as soon as buckets are no wider than half a column, here 36 s [3].

Suppose $\mathcal{C}$ holds one-minute buckets from an earlier coarse read of this region. A one-minute bucket (60 s) is wider than the 36 s half-column, so it can straddle a column boundary and leave that column with no fully-contained bucket, making $\mathcal{E}(\text{view})$ uncomputable. These one-minute buckets cannot be sharpened from what is cached, since a finer bucket’s min and max are not recoverable

Table 1: The twelve cached 1-minute buckets ($\alpha = 2$) tiling the ramp segment σ of q_1 (normalized power).

Span (min)	count	min	max	sum
[0, 1)	4	0.25	0.35	1.20
[1, 2)	4	0.30	0.40	1.40
[2, 3)	4	0.35	0.45	1.60
[3, 4)	4	0.40	0.50	1.80
[4, 5)	4	0.45	0.55	2.00
[5, 6)	4	0.50	0.60	2.20
[6, 7)	4	0.55	0.65	2.40
[7, 8)	4	0.60	0.70	2.60
[8, 9)	4	0.65	0.75	2.80
[9, 10)	4	0.70	0.80	3.00
[10, 11)	4	0.75	0.85	3.20
[11, 12)	4	0.80	0.90	3.40

from a coarser one, so VASTA *fetches* the affected columns from the store at the finer calendar level the global $\alpha = 2$ selects (the 1.2 min column subdivides to 36 s, floored to 30 s). The two 30 s buckets it fetches beneath a one-minute bucket $B_v = (\text{min } 0.60, \text{max } 0.70, \text{count } 6, \text{sum } 3.90)$,

$$(0.60, 0.65, 3, 1.92) \quad \text{and} \quad (0.65, 0.70, 3, 1.98),$$

roll up to B_v exactly—counts $3 + 3 = 6$, sums $1.92 + 1.98 = 3.90$, and min, max reducing to 0.60, 0.70—so they nest in the calendar grid $\mathcal{C}$ already uses. Each 30 s sub-bucket is no wider than the 36 s half-column, so it lies fully inside a column; every column now holds a fully-contained bucket, $\mathcal{E}(\text{view})$ is computable, and $\mathcal{E}(\text{view}) \leq \rho_V$. The 30 s buckets stay in $\mathcal{C}$ for the search that follows.

1.3 Search: a pattern query

Having seen a rise-to-rated event on the chart, the analyst marks it and searches the past week for it. The marked shape is a *pattern* of three segment specifications, $Q_P = \langle q_1, q_2^{[1,3]}, q_3^* \rangle$: a power ramp q_1 (angle $\phi_1 = [6^\circ, 11^\circ]$, duration 3–5 time units), a near-flat plateau q_2 at rated output (angle $\phi_2 = [-1^\circ, 1^\circ]$, duration 1–2 units) repeated one to three times, and a trailing decline q_3 (angle $\phi_3 = [-11^\circ, -6^\circ]$, duration 3–5 units) under Kleene closure (zero or more). The query is posed over the past week $[t'_s, t'_e)$ at time unit $u = 3$ min and tolerance $\rho_P = 0.1$, with the default aggregation factor $\alpha = 2$: halving the 3-minute unit gives 90 s, which floors to the 1-minute calendar level, so the pattern side reads the week as 1-minute buckets.

The searched week is mostly new ground: the twelve-hour window the analyst just rendered is a small slice of it, already in $\mathcal{C}$ at 30 s, while the surrounding days have not been touched and are absent from $\mathcal{C}$. Over the rendered window the search fetches nothing, since the cached 30 s view buckets roll up to the 1-minute buckets it reads. Over the untouched days VASTA issues one grouped fetch at the 1-minute level and folds the returned buckets into $\mathcal{C}$, where they stay. The ε -NFA then scans $\mathcal{S}^u$ for the pattern; consider the ramp segment q_1 of one candidate event found in the untouched days, a twelve-minute σ tiled by the twelve cached 1-minute buckets of Table 1; we write time in units of $u = 3$ min, so bucket i spans $[\frac{i}{3}, \frac{i+1}{3})$ with width $w_i = \frac{1}{3}$ and midpoint m_i .

Plugging these aggregates into §3 gives $N = 48$, $\hat{t} = 2.0$, $\hat{v} = 0.575$, and the numerator and denominator intervals

$$S_{tv} \in [7.5, 11.5], \quad S_{tt} \in [47.6, 86.2].$$

Since $S_{tt}^- = 47.6 > 0$, the four corner ratios bracket the slope,

$$\beta(\sigma) \in \left[\frac{7.5}{86.2}, \frac{11.5}{47.6} \right] \approx [0.09, 0.24],$$

i.e. an angle interval $[\theta^-, \theta^+] = [5.0^\circ, 13.6^\circ]$ (the midpoint estimate $\hat{\beta} \approx 0.15$ or 8.5° lies inside it). This straddles both edges of the ramp filter $\phi_1 = [6^\circ, 11^\circ]$: the fraction lying outside is the overshoot $o(\sigma) \approx 0.42$, far above $\rho_P = 0.1$, so at the default $\alpha = 2$ the segment is *ambiguous*. Its region is exactly what refinement will touch.

1.4 Refine: resolving the ambiguous region

VASTA refines only σ 's region, to 30-second buckets: α is doubled, and is now 4, halving the 1-minute reads to the 30s calendar level, two nesting in each 1-minute bucket. Over σ , in the newly fetched days, it fetches these; had σ fallen in the just-rendered window, already cached at 30s, they would already sit in $\mathcal{C}$ —the search's refined level is exactly the view's. The split is exact: the $[4, 5)$ bucket $(0.45, 0.55, 4, 2.0)$ becomes

$$(0.45, 0.50, 2, 0.96) \quad \text{and} \quad (0.50, 0.55, 2, 1.04),$$

which roll up to it (counts $2 + 2 = 4$, sums $0.96 + 1.04 = 2.0$, extremes reduced to $0.45, 0.55$). Recomputing the enclosure over the twenty-four 30-second buckets—whose slack shrinks with the halved widths—tightens the interval to

$$[\theta^-, \theta^+] = [6.8^\circ, 10.6^\circ] \quad (\beta(\sigma) \approx [0.12, 0.19]),$$

now *fully inside* ϕ_1 , so the overshoot falls to $o(\sigma) = 0$ and the ramp clears its filter.

The pattern's other segments are bounded and tested the same way: the one-to-three plateau segments q_2 (near-flat, $\phi_2 = [-1^\circ, 1^\circ]$) and the optional decline q_3 each fall within their own filters here. The match's exposure then combines the per-segment overshoots, $o(M) = 1 - \prod_{\sigma \in M} (1 - o(\sigma))$; with every segment inside its filter, $o(M) \leq \rho_P$ and the event is reported as a *confident* match. Had $o(M)$ still exceeded ρ_P , the loop would refine once more or, as a last resort, resolve the region against exact data; here one step suffices.

Two directions of reuse close the loop, and the resolutions decide what each buys. The seen window's 30s view buckets served this search for free, rolling up to its 1-minute reads. The other way, the 1-minute buckets the search fetched over the untouched days stay in $\mathcal{C}$: they are too coarse for the twelve-hour zoom, whose 1.2-minute columns still need 30s buckets, but they are exactly what a week-scale overview needs, since on the same 600-column canvas a week spans 16.8-minute columns and a 1-minute bucket sits fully inside every one. So the overview the analyst renders next to locate the week's matches is served from $\mathcal{C}$ within ρ_V with no new fetch; and where refinement already produced 30s buckets over σ , even a return to the twelve-hour zoom of that region is served at no cost, since the search's refined level is the view's own. Either way, both the picture and the match are read from the *same* four aggregates— $\mathcal{E}(\text{view}) \leq \rho_V$ on the render, and a slope interval that provably encloses the true slope on the search—the single substrate the paper argues for, shown here end to end on one series.

2 Motivation & Problem Formulation

For the reviewers: This is Section III of the paper, with improvements.

2.1 Motivation

Seeing and searching interleave over one series, but they handle their work very differently. On the seeing side, the state of the art is a session-persistent cache: MinMaxCache [3] keeps min/max summaries across the session and serves each view from them at interactive latency, within a guaranteed pixel-error bound, reusing them for every later view over the same region. The pattern side has no analog: search tests a segment’s slope, conventionally that of an ordinary least-squares (OLS) fit [4], and recomputes it from raw data at every query. A cache could store what the slope needs, but doing so means carrying the full regression sums, well beyond the two extrema rendering keeps, enlarging a summary that error-bounded rendering keeps small precisely to stay interactive.

We propose that the loop closes only if one cache does for search what MinMaxCache does for the view—resolves each query at interactive latency to the accuracy it is asked for—without giving up the rendering it already serves, and by adding to the cached extrema only a couple of cheap aggregates rather than the exact regression state.

2.2 Setting and Notation

We work with a univariate time series $\mathcal{S} = \{(t_i, v_i)\}_{i=1}^N$ over $T = [t_{\text{start}}, t_{\text{end}})$, stored remotely and too large to scan on every interaction. The analyst sees and searches $\mathcal{S}$ through a caching middleware that holds summaries between the frontend and the store. Both operations are scoped to a time range $[t_s, t_e) \subseteq T$, indicating the window on screen for a view and the stretch being searched for a pattern.

The shared summary The cache divides time into *buckets* aligned to calendar boundaries (a minute, an hour, a day). A bucket b spans $\tau_b = [t_s^b, t_e^b)$ and summarises the points falling in it by four aggregates,

$$\text{agg}(b) = (\min_b, \max_b, \text{count}_b, \text{sum}_b),$$

with no assumption about where the points sit in time within τ_b . Calendar alignment sorts buckets into *levels* or *time units*—standard spans from milliseconds up to days—that nest by whole multiples, so a coarser bucket is the exact union of the finer ones inside it; an hour, for instance, contains twelve five-minute buckets and never splits one. Such a coarser bucket *rolls up* the aggregates of its finer parts—count and sum add, min and max reduce—so any level assembles from a finer one that nests in it without revisiting the data. The cache $\mathcal{C}$ is the set of buckets, at possibly different levels over different regions, materialized across a querying session.

Seeing A *view-query* $Q_V = \langle \tau, VP, \rho_V \rangle$ renders a range $\tau = [t_s, t_e)$ onto a viewport $VP = W \times H$ pixel (the visualization canvas). A *view* is the set of pixels turned on from the query (either the exact set, if the query is answered directly from the raw data, or a bounded approximation if answered via the view). From the cached buckets alone, one can bound the fraction of pixels that may differ from the exact render by using the rendering and error calculation used by MinMaxCache [3]. We write $\mathcal{E}(\text{view})$ for this bounded pixel-difference fraction. The query carries a *tolerance* $\rho_V \in [0, 1]$, and is served correctly when $\mathcal{E}(\text{view}) \leq \rho_V$, which holds once each column contains a fully-contained bucket [3].

Table 2: Notation used in this paper

$\mathcal{S}, T$	series of length N ; its time range
Q_V, Q_P	view-query and pattern-query
ρ_V, ρ_P	tolerance of a view / pattern-query
$\mathcal{E}(\text{view}), \mathcal{E}(\text{pattern})$	view pixel-error / pattern error estimate
$[t_s, t_e)$	time range of a view or pattern-query
W, H	viewport width (columns), height (rows)
b, τ_b	bucket and its calendar span
u	time unit (a calendar level) of a pattern-query
$\mathcal{S}^u$	the matched over series of u -sized buckets
$\text{agg}(b)$	$(\min_b, \max_b, \text{count}_b, \text{sum}_b)$
$\mathcal{C}$	cache: materialised buckets
α	aggregation factor (sub-buckets per unit)
$\langle q_1, \dots, q_k \rangle$	pattern: sequence of segment specifications
μ	repetition factor of a specification
ϕ_j^T, ϕ_j^Θ	duration and angle constraints of q_j
$\sigma, \beta(\sigma)$	segment matching q_j , and its OLS slope
$[\beta^-(\sigma), \beta^+(\sigma)]$	enclosing slope interval from $\mathcal{C}$
$o(\sigma), o(M)$	segment / match overshoot

Searching A *pattern-query* $Q_P = \langle \tau, u, P, \rho_P \rangle$ is posed over a range $\tau = [t_s, t_e)$ at a *time unit* u at which the time range divides into consecutive u -sized buckets, producing the sequence $\mathcal{S}^u$ over which the query is matched. A *pattern* P is a sequence of *segment specifications* $\langle q_1, \dots, q_k \rangle$, each under a repetition factor μ —a fixed count, a range $[p, q]$, or Kleene closure (zero or more) [4]; a specification may itself be a sub-pattern, so μ ranges over a group as well as a single segment. Each segment specification q_j carries a duration constraint ϕ_j^T (e.g. two to three time units) and an angle constraint $\phi_j^\Theta = [\theta_j^-, \theta_j^+]$ (e.g. between 10° and 45°). A *segment* σ —composed of one or more consecutive buckets—satisfies q_j when its duration (number of buckets) meets ϕ_j^T and the OLS slope of its points,

$$\beta(\sigma) = \frac{\sum_{t_i \in \sigma} (t_i - \bar{t})(v_i - \bar{v})}{\sum_{t_i \in \sigma} (t_i - \bar{t})^2},$$

lies in the slope range $R_j = [\tan \theta_j^-, \tan \theta_j^+]$. A *candidate* is a contiguous stretch of the series partitioned into consecutive segments; it is a *match* when that sequence of segments conforms to the pattern—each specification satisfied as many times as its repetition operator allows.

Unlike the view, search has no cached quantity for the slope it needs; $\beta(\sigma)$ is not in the cache, and computing it from the data is the cost we set out to avoid. The query carries a *tolerance* $\rho_P \in [0, 1]$. VASTA grades the returned matches by $\mathcal{E}(\text{pattern})$, a heuristic that estimates the fraction of them exact data would overturn rather than bounding it, and answers the query once $\mathcal{E}(\text{pattern}) \leq \rho_P$. The view can bound its answer outright, whereas search can only grade its own; the two produce different kinds of answers, something we explore further in Section 2.3 and in Section IV-C of the paper.

2.3 Problem Formulation

The problem is to answer a session of view and patter queries from the same cache, as fast as possible, without sacrificing quality.

Problem 1 (Serving an interleaved see-search session). *Given a session of queries $\mathbf{Q_S}$ including view and pattern queries (as previously defined) Q_i over a time series $\mathcal{S}$ for the time domain T minimize the cumulative query time of the entire session such that (a) every view keeps its pixel-difference fraction $\mathcal{E}(\text{view})$ within ρ_V of the exact render, and, (b) every pattern query keeps its estimated error $\mathcal{E}(\text{pattern})$ within ρ_P .*

Both tolerances play the same role: each is the accuracy a query asks of its answer; refinement works the answer toward it, and the system fetches only as much as meeting it requires. Cumulative time is dominated by backend fetches: answers drawn entirely from $\mathcal{C}$ are immediate, while every miss pays the round-trip plus the cost of the data it pulls. The cache’s job is to make hits frequent and misses cheap.

2.4 The slope problem

The view side is sourced from MinMaxCache; the open problem is search, and its difficulty has one source.

Observation 1 (The summary does not determine the slope). *Two series with identical (min, max, count, sum) over the buckets covering a segment can have different OLS slopes, since $\beta(\sigma)$ depends on where points fall in time within each bucket, which the four aggregates discard.*

For instance, a bucket on $[0, 1)$ with min=2, max=8, count=2, sum=10 admits the two-point placement $(0.1, 2), (0.9, 8)$ at slope +7.5, and equally $(0.1, 8), (0.9, 2)$ at slope −7.5. The same aggregates produce opposite slopes. Recovering $\beta(\sigma)$ exactly would require storing the regression sums an OLS fit needs, involving too many values beyond what the extrema rendering keeps. We instead *bound* the slope, computing from the cached aggregates a point estimate together with an interval guaranteed to contain the true OLS slope of the segment’s raw points.

Definition 1 (Slope-bounding cache). *A cache is slope-bounding if, for every covered segment σ , it returns from the cached spans and aggregates alone an interval $[\beta^-(\sigma), \beta^+(\sigma)]$ containing $\beta(\sigma)$ under every placement of points consistent with the summaries.*

2.5 Similarities and differences of pattern and view queries, all served by the same cache

Similarities Each query, view or pattern, is answered from the cached aggregates against its own bound—an exact pixel bound for the view, the enclosing slope interval for each segment of a pattern—and where a query’s tolerance is not yet met, the cache is refined to finer sub-buckets until it is. Crucially, the two refine the *same* sub-buckets, so work done for one is available to the other. We take the min/max rendering and its pixel-error bound from the visualization cache of [3], which, when a view exceeds its tolerance, refetches the whole window at a finer granularity; here refinement is instead per-region and iterative—only the parts still over the tolerance are refined—and the same mechanism serves search, so a session’s refinements accumulate in $\mathcal{C}$ across both operations.

Differences The machinery is symmetric; what the two tolerances can promise is not, and the difference is in the kind of answer each produces. A view query’s answer is a *representation*: a fixed grid of pixel columns set by the viewport, not by the data, so per-column bounds tile into a guaranteed image bound. A pattern query’s answer is a *found set*. The unit beneath it is bounded

just as tightly—the interval of Definition 1 encloses each segment’s true slope—but the answer is not built by tiling them. Which segments become matches is the query’s decision, settled jointly by duration, angle, and the repetitions the pattern allows. Nonetheless, a bound on a segment is no fragment of a bound on the set.

Since every true segment’s interval contains its true slope, no true match is ever dropped and the returned set is complete; but a straddling interval can admit a segment exact data would reject, so the set may carry spurious matches, degrading precision. This residue is what $\mathcal{E}(\text{pattern})$ estimates and what refinement drives down, until $\mathcal{E}(\text{pattern}) \leq \rho_P$ (Section IV-C of the paper). The view meets ρ_V as a guaranteed bound; search meets ρ_P as a target it is driven toward.

3 Bounding the OLS Slope from Bucket Summaries

For the reviewers: This section backs the slope-enclosure result of the paper (Theorem IV.1 / Definition 1). It has two parts: (a) the intuition of the enclosure theorem with an intuitive proof sketch (§3.1), which is a stand-alone subsection, to appear in the paper as Section IV.B, and, (b) the full derivation of the proof with all the details (§3.2).

3.1 Bounding the slope: intuition and proof sketch

Assume a segment σ tiled by cached buckets $b_1, \dots, b_n$, where each b_i spans $[s_i, e_i)$ with width w_i and midpoint m_i , and holds c_i points with extremes ℓ_i, h_i and exact value-mean μ_i . The segment's OLS slope is $\beta(\sigma) = S_{tv}/S_{tt}$ with

$$S_{tv} = \sum_{t_i \in \sigma} (t_i - \bar{t})(v_i - \bar{v}), \quad S_{tt} = \sum_{t_i \in \sigma} (t_i - \bar{t})^2.$$

By Observation 1, the aggregates do not fix $\beta(\sigma)$, but confine it to an interval which is what we compute.

Since each bucket stores its mean μ_i , its value-centroid is computed exactly. A bucket's time-centroid $\bar{t}_i$, however, is unknown; the best estimation we can offer is that it lies within half a width of the midpoint, $|\bar{t}_i - m_i| \leq w_i/2$. Approximating each unknown time-centroid in with its midpoint and each unknown value-spread with the range $[\ell_i, h_i]$ confines S_{tv} and S_{tt} to intervals whose ends are computable estimates with worst-case error proportional to bucket width w_i :

$$S_{tv} \in [S_{tv}^-, S_{tv}^+], \quad S_{tt} \in [S_{tt}^-, S_{tt}^+],$$

The slope $\beta(\sigma) = S_{tv}/S_{tt}$ is their ratio; when $S_{tt}^- > 0$ it is enclosed by the four corner ratios of those two intervals,

$$\beta(\sigma) \in \left[\min_{\pm} \frac{S_{tv}^{\pm}}{S_{tt}^{\pm}}, \max_{\pm} \frac{S_{tv}^{\pm}}{S_{tt}^{\pm}} \right] = [\beta^-(\sigma), \beta^+(\sigma)], \quad (1)$$

whose arctan gives the angle interval $[\theta^-(\sigma), \theta^+(\sigma)]$ the matcher tests. The one caveat is the sign of S_{tt}^- : even though the true S_{tt} , a sum of squares, is always positive, S_{tt}^- is a computable estimate $\hat{S}_{tt}$ minus an error $E \downarrow$. $\hat{S}_{tt}$ measuring the distance of a bucket's midpoint from the segment's midpoint can be smaller than $E \downarrow$ with narrow segments, hence too few involved buckets. Then, VASTA reports the slope unbounded.

Theorem 1 (Slope enclosure). *For a segment σ tiled by cached buckets with $S_{tt}^- > 0$, the interval $[\beta^-(\sigma), \beta^+(\sigma)]$ of (1), computed from the four aggregates alone, contains the true OLS slope $\beta(\sigma)$ under every placement of points consistent with those aggregates.*

Proof sketch. The aggregates admit exactly the point clouds that, in each bucket, place c_i points with values in $[\ell_i, h_i]$ of mean μ_i and times in $[s_i, e_i)$. Over any such cloud both S_{tv} and S_{tt} split exactly into a between-bucket part (where the bucket centroids sit) and a within-bucket part (the spread inside each bucket). The only unknown is the time placement, and the midpoint inequality $|\bar{t}_i - m_i| \leq w_i/2$ cancels its first-order effect on the between part and caps what is left by a term proportional to w_i ; the value range $[\ell_i, h_i]$ caps the within part likewise. This is what places S_{tv} and S_{tt} in the two intervals above. A ratio is monotone in each argument on a rectangle whose denominator keeps one sign, so its extremes over $[S_{tv}^-, S_{tv}^+] \times [S_{tt}^-, S_{tt}^+]$ fall at the corners; the true pair lies somewhere in that rectangle, so its ratio lies between the smallest and largest corner ratio, which is (1). □

The theorem discharges the slope-bounding contract of Definition 1, and its standing consequence is *refinability*: every error scales with a bucket width, so replacing a bucket by the finer level beneath it shrinks both intervals, lifts S_{tt}^- toward positive, and tightens the slope toward its exact value—the lever the refinement loop pulls.

3.2 Full proof

The derivation below carries out the sketch with every constant, in the same t, v notation. It builds each bound as a computable estimate plus an error bar: $S_{tv}^\pm = \hat{S}_{tv} \pm E_{tv}$ for the numerator, and $S_{tt}^- = \hat{S}_{tt} - E_\downarrow$, $S_{tt}^+ = \hat{S}_{tt} + E_\downarrow + E_\uparrow$ for the denominator.

3.2.1 The problem, concretely

We have a time series of raw points (t_{ij}, v_{ij}) . We want one number: the ordinary least-squares (OLS) slope of a contiguous stretch of that series, the segment. Written out, that slope is

$$\beta = \frac{S_{tv}}{S_{tt}}, \quad S_{tt} = \sum_{i,j} (t_{ij} - \bar{t})^2, \quad S_{tv} = \sum_{i,j} (t_{ij} - \bar{t})(v_{ij} - \bar{v}),$$

where $\bar{t}, \bar{v}$ are the means of all the points in the segment.

What we actually have. We do *not* have the raw points. The store has already chopped the segment into n consecutive *buckets* and thrown the raw points away, keeping for each bucket i only a five-number summary:

- its time span $[s_i, e_i]$ (exact calendar boundaries),
- how many points it holds, c_i ,
- the smallest and largest v in it, ℓ_i and h_i ,
- the exact mean of its v values, μ_i .

That is everything. We never see an individual t_{ij} or v_{ij} again.

What we want to produce. Because the raw points are gone, we cannot compute β exactly – many different point clouds are consistent with the same bucket summaries, and they have different slopes. So instead of a single number we will produce an **interval** $[\beta^-, \beta^+]$ that is *guaranteed* to contain the true slope, whatever the (unknown) raw points were, as long as they are consistent with the summaries. The whole proof is the construction of that interval and the argument that it really does trap β .

The one source of uncertainty. For a single bucket, the summary pins down a lot but not everything. The v side is well controlled: every v_{ij} lies in $[\ell_i, h_i]$, and we know the bucket’s exact v -mean μ_i . The t side is the problem. We know every t_{ij} lies in the time span $[s_i, e_i]$, but we have *no* information about where inside that span the points fall. They could be bunched at the left edge, bunched at the right, spread out—the summary does not say. This single unknown—the horizontal placement of points inside each bucket—is the entire source of slack in everything that follows. Keep an eye on it.

3.2.2 Setting up the bookkeeping

Fix notation per bucket i :

$$m_i = \frac{s_i + e_i}{2} \quad (\text{the midpoint}), \quad w_i = e_i - s_i \quad (\text{the width}), \quad N = \sum_i c_i \quad (\text{total points}).$$

The true (but unknown) average t -position of the points in bucket i is

$$\bar{t}_i = \frac{1}{c_i} \sum_j t_{ij}.$$

We do not know $\bar{t}_i$. All the summary tells us is that it lies somewhere in the bucket's span $[s_i, e_i]$.

Why the midpoint—a minimax choice. To compute anything we must pick a single representative t -position for bucket i to stand in for the unknown $\bar{t}_i$. Call our choice $p_i \in [s_i, e_i]$. Whatever we pick, the error we will carry is $\bar{t}_i - p_i$, and an adversary—choosing the worst point cloud consistent with the summary—may place the true $\bar{t}_i$ anywhere in $[s_i, e_i]$. So the worst-case size of our error is

$$\max_{\bar{t}_i \in [s_i, e_i]} |\bar{t}_i - p_i| = \max(p_i - s_i, e_i - p_i).$$

We get to pick p_i *before* the adversary picks $\bar{t}_i$, so we should pick the p_i that makes this worst case as small as possible. The maximum of the two distances $p_i - s_i$ and $e_i - p_i$ is smallest exactly when they are equal,

$$p_i - s_i = e_i - p_i \quad \Longleftrightarrow \quad p_i = \frac{s_i + e_i}{2} = m_i,$$

i.e. the **midpoint**. Any other point in the bucket makes one of the two distances longer, and the adversary simply places $\bar{t}_i$ at that far end, giving a strictly larger guaranteed error. The midpoint is therefore not a convenience—it is the unique choice that minimizes the worst-case t -error, and with it that worst case is exactly half the bucket width:

$$\delta_i := \bar{t}_i - m_i, \quad |\delta_i| \leq \frac{w_i}{2}. \quad (\star)$$

Inequality $(\star)$ is the *only* thing we know about the t -placement, and the whole bound is built by pushing $(\star)$ through the algebra. Because the midpoint achieves the smallest possible constant $(w_i/2)$ on the right of $(\star)$, every error term that later inherits this constant— E_{tv} , $E_\downarrow$, $E_\uparrow$ —is as tight as the available information allows. Pick any other representative and all of them grow.

The v side needs no such choice: the summary stores the *exact* bucket mean μ_i , so the bucket's v -centroid is exactly μ_i (write $\bar{v}_i = \mu_i$)—an equality, not a bound.

Quantities we can actually compute. We cannot compute $\bar{t}$ (it needs the unknown $\bar{t}_i$). But with the midpoint as each bucket's t -representative, and the exact mean as each bucket's v -representative, we can compute the count-weighted surrogates

$$\hat{t} = \frac{1}{N} \sum_i c_i m_i, \quad \hat{v} = \frac{1}{N} \sum_i c_i \mu_i.$$

These are our stand-ins for the unknown true means $\bar{t}, \bar{v}$. (A second, technical reason the midpoint is convenient: it makes the sums $\sum_i c_i(\mu_i - \hat{v})$ and $\sum_i c_i u_i$ collapse to zero, which is what kills the first-order error terms in Lemmas B and C. That cancellation actually works for any affine choice of representative; the minimax argument above is the reason the midpoint in particular is the *right* affine choice.) The first job is to measure how wrong the stand-ins are.

3.2.3 Step 1—the means (Lemma A)

Lemma 1. $\bar{v} = \hat{v}$ exactly. And the t -mean error, $\Delta := \bar{t} - \hat{t}$, satisfies

$$\Delta = \frac{1}{N} \sum_i c_i \delta_i, \quad |\Delta| \leq \frac{1}{N} \sum_i c_i \frac{w_i}{2} =: D.$$

Proof. Group the global v -average by bucket. The points in bucket i contribute total v equal to $c_i \bar{v}_i = c_i \mu_i$, so

$$\bar{v} = \frac{1}{N} \sum_{i,j} v_{ij} = \frac{1}{N} \sum_i c_i \bar{v}_i = \frac{1}{N} \sum_i c_i \mu_i = \hat{v}.$$

This is exact—nothing was approximated, because μ_i is the exact bucket mean. The v side carries no slack.

For t , do the same grouping but substitute $\bar{t}_i = m_i + \delta_i$ (the definition of δ_i):

$$\bar{t} = \frac{1}{N} \sum_i c_i \bar{t}_i = \frac{1}{N} \sum_i c_i (m_i + \delta_i) = \underbrace{\frac{1}{N} \sum_i c_i m_i}_{=\hat{t}} + \frac{1}{N} \sum_i c_i \delta_i.$$

So $\bar{t} - \hat{t} = \frac{1}{N} \sum_i c_i \delta_i$, which is the stated Δ . Bounding it: by the triangle inequality and $(\star)$,

$$|\Delta| \leq \frac{1}{N} \sum_i c_i |\delta_i| \leq \frac{1}{N} \sum_i c_i \frac{w_i}{2} = D. \quad \square$$

Plain meaning: our v stand-in is perfect; our t stand-in is off by an unknown amount Δ , but that amount is provably no larger than D , a number we can compute from widths and counts alone.

3.2.4 Step 2—separating “between” and “within”

To bound S_{tt} and S_{tv} we split each into two parts: variation *between* the buckets (driven by where the bucket centroids sit) and variation *within* each bucket (driven by the spread of points around their own centroid). This split is exact.

For any bucket the points satisfy two zero-sum identities by definition of the centroid:

$$\sum_j (t_{ij} - \bar{t}_i) = 0, \quad \sum_j (v_{ij} - \bar{v}_i) = 0.$$

Write $t_{ij} - \bar{t} = (\bar{t}_i - \bar{t}) + (t_{ij} - \bar{t}_i)$ and similarly for v , then expand the squares/products in S_{tt}, S_{tv} . Every cross term contains a factor $\sum_j (t_{ij} - \bar{t}_i)$ or $\sum_j (v_{ij} - \bar{v}_i)$, which is zero, so the cross terms vanish and we are left with exactly:

$$S_{tt} = \underbrace{\sum_i c_i (\bar{t}_i - \bar{t})^2}_{\text{between}} + \underbrace{\sum_i A_i}_{\text{within}}, \quad A_i = \sum_j (t_{ij} - \bar{t}_i)^2,$$

$$S_{tv} = \underbrace{\sum_i c_i (\bar{t}_i - \bar{t}) (\mu_i - \bar{v})}_{\text{between}} + \underbrace{\sum_i B_i}_{\text{within}}, \quad B_i = \sum_j (t_{ij} - \bar{t}_i) (v_{ij} - \bar{v}_i).$$

Both identities are exact. Now we bound the two parts separately: the between part using Lemma 1 (it involves the unknown $\bar{t}$, which we replace by $\hat{t}$ at the cost of Δ), and the within part using the bucket's min/max and width.

3.2.5 Step 3—bounding S_{tv} (Lemma B)

Define the quantity we *can* compute—the between part with the unknown true means replaced by the computable stand-ins:

$$\hat{S}_{tv} = \sum_i c_i (m_i - \hat{t}) (\mu_i - \hat{v}).$$

Lemma 2. $|S_{tv} - \hat{S}_{tv}| \leq E_{tv}$, where, writing $d_i = \max(h_i - \mu_i, \mu_i - \ell_i)$ for the largest possible v -deviation in bucket i ,

$$E_{tv} = \sum_i c_i |\mu_i - \hat{v}| \frac{w_i}{2} + \sum_i c_i w_i d_i.$$

Proof. Start from the exact between part of S_{tv} and replace the unknown $\bar{t}_i - \bar{t}$ by computable pieces plus error. Using $\bar{t}_i = m_i + \delta_i$ and $\bar{t} = \hat{t} + \Delta$ (Lemma A),

$$\bar{t}_i - \bar{t} = (m_i - \hat{t}) + (\delta_i - \Delta).$$

Also $\bar{v} = \hat{v}$ exactly (Lemma A), so $\mu_i - \bar{v} = \mu_i - \hat{v}$. Substitute into the between sum:

$$\sum_i c_i (\bar{t}_i - \bar{t}) (\mu_i - \bar{v}) = \sum_i c_i [(m_i - \hat{t}) + (\delta_i - \Delta)] (\mu_i - \hat{v}).$$

Distribute into three groups:

$$= \underbrace{\sum_i c_i (m_i - \hat{t}) (\mu_i - \hat{v})}_{=\hat{S}_{tv}} + \sum_i c_i \delta_i (\mu_i - \hat{v}) - \Delta \sum_i c_i (\mu_i - \hat{v}).$$

The last group vanishes. Why: Δ does not depend on i , so pull it out, and

$$\sum_i c_i (\mu_i - \hat{v}) = \left(\sum_i c_i \mu_i \right) - \hat{v} \sum_i c_i = N \hat{v} - \hat{v} N = 0,$$

using the definition $\hat{v} = \frac{1}{N} \sum_i c_i \mu_i$. So the entire Δ term is $-\Delta \cdot 0 = 0$. This is the key cancellation: the error in our t -mean stand-in does *not* corrupt the between part of S_{tv} at first order.

What remains, after adding back the within part $\sum_i B_i$:

$$S_{tv} - \hat{S}_{tv} = \underbrace{\sum_i c_i \delta_i (\mu_i - \hat{v})}_{\text{(I): unknown } t\text{-placement}} + \underbrace{\sum_i B_i}_{\text{(II): within-bucket spread}}.$$

Bound each piece using only what the summaries give us.

Piece (I). The only unknown is δ_i , and $(\star)$ says $|\delta_i| \leq w_i/2$. Everything else is computable. So

$$\left| \sum_i c_i \delta_i (\mu_i - \hat{v}) \right| \leq \sum_i c_i |\mu_i - \hat{v}| |\delta_i| \leq \sum_i c_i |\mu_i - \hat{v}| \frac{w_i}{2}.$$

That is exactly the first sum in E_{tv} .

Piece (II). Inside bucket i , bound the product term by term. Both t_{ij} and $\bar{t}_i$ lie in the span $[s_i, e_i)$, so their difference is at most the width: $|t_{ij} - \bar{t}_i| \leq w_i$. And every v_{ij} lies in $[\ell_i, h_i]$ while $\bar{v}_i = \mu_i$, so $|v_{ij} - \mu_i| \leq \max(h_i - \mu_i, \mu_i - \ell_i) = d_i$. Therefore

$$|B_i| \leq \sum_j |t_{ij} - \bar{t}_i| |v_{ij} - \mu_i| \leq \sum_j w_i d_i = c_i w_i d_i,$$

and summing over buckets gives the second sum in E_{tv} .

Adding the two bounds via the triangle inequality yields $|S_{tv} - \hat{S}_{tv}| \leq E_{tv}$. $\square$

Plain meaning: $\hat{S}_{tv}$ is our computable estimate of S_{tv} . It is wrong by at most E_{tv} , and E_{tv} has two clearly identified causes—not knowing the horizontal placement (piece I) and not knowing how points scatter inside a bucket (piece II). Both are controlled purely by widths, counts, and the stored min/max.

3.2.6 Step 4—bounding S_{tt} (Lemma C)

Now the denominator. Define the computable estimate

$$\hat{S}_{tt} = \sum_i c_i u_i^2, \quad u_i := m_i - \hat{t}.$$

Lemma 3. $\hat{S}_{tt} - E_{\downarrow} \leq S_{tt} \leq \hat{S}_{tt} + E_{\downarrow} + E_{\uparrow}$, where

$$E_{\downarrow} = \sum_i c_i |m_i - \hat{t}| w_i, \quad E_{\uparrow} = \sum_i c_i \left(\frac{w_i}{2} + D \right)^2 + \sum_i c_i \frac{w_i^2}{4}.$$

Proof. Take the exact between part of S_{tt} and substitute $\bar{t}_i - \bar{t} = u_i + e_i$ where $e_i := \delta_i - \Delta$ collects all the t -placement uncertainty. Expanding the square:

$$\sum_i c_i (\bar{t}_i - \bar{t})^2 = \sum_i c_i u_i^2 + 2 \sum_i c_i u_i e_i + \sum_i c_i e_i^2.$$

The first sum is $\hat{S}_{tt}$. In the cross sum, replace e_i by $\delta_i - \Delta$; the $-\Delta$ piece drops out because $\sum_i c_i u_i = \sum_i c_i (m_i - \hat{t}) = N\hat{t} - \hat{t}N = 0$ (same kind of cancellation as before). So $\sum_i c_i u_i e_i = \sum_i c_i u_i \delta_i$. Putting the within part $\sum_i A_i$ back:

$$S_{tt} = \hat{S}_{tt} + 2 \sum_i c_i u_i \delta_i + \underbrace{\sum_i c_i e_i^2}_{\geq 0} + \underbrace{\sum_i A_i}_{\geq 0}. \quad (\dagger)$$

The last two sums are sums of squares, hence ≥ 0 . The middle (signed) term is controlled by $(\star)$:

$$\left| 2 \sum_i c_i u_i \delta_i \right| \leq 2 \sum_i c_i |u_i| |\delta_i| \leq 2 \sum_i c_i |m_i - \hat{t}| \frac{w_i}{2} = \sum_i c_i |m_i - \hat{t}| w_i = E_{\downarrow}.$$

Lower bound. In (†), the two squared sums only add to S_{tt} , so dropping them can only decrease the right-hand side:

$$S_{tt} \geq \widehat{S}_{tt} + 2 \sum_i c_i u_i \delta_i \geq \widehat{S}_{tt} - E_{\downarrow}.$$

Upper bound. Bound the two non-negative sums from above.

For $\sum_i c_i e_i^2$: $e_i = \delta_i - \Delta$, and $|\delta_i| \leq w_i/2$ by (★) while $|\Delta| \leq D$ by Lemma A, so $|e_i| \leq w_i/2 + D$ and $e_i^2 \leq (w_i/2 + D)^2$. Hence $\sum_i c_i e_i^2 \leq \sum_i c_i (w_i/2 + D)^2$, the first sum in $E_{\uparrow}$.

For $\sum_i A_i$: $A_i = \sum_j (t_{ij} - \bar{t}_i)^2$ is the total squared spread of c_i points confined to an interval of width w_i . That spread is largest when the points are pushed as far apart as the interval allows, i.e. split between the two endpoints; a short calculation shows the maximum is $c_i w_i^2/4$. So $A_i \leq c_i w_i^2/4$ and $\sum_i A_i \leq \sum_i c_i w_i^2/4$, the second sum in $E_{\uparrow}$.

Substituting both into (†) and using $|2 \sum c_i u_i \delta_i| \leq E_{\downarrow}$:

$$S_{tt} \leq \widehat{S}_{tt} + E_{\downarrow} + E_{\uparrow}. \quad \square$$

Plain meaning: $\widehat{S}_{tt}$ is our computable estimate of S_{tt} . Unlike S_{tv} , the uncertainty here is one-sided in character—the unknown placement and the within-bucket spread can only *inflate* S_{tt} relative to the estimate (that is why the upper error $E_{\uparrow}$ is separate and larger), while the lower side is controlled by the single signed term $E_{\downarrow}$.

3.2.7 Step 5—from the two boxes to a slope interval

We now know, with certainty,

$$\begin{aligned} S_{tv} \in [S_{tv}^-, S_{tv}^+], \quad S_{tv}^- &= \widehat{S}_{tv} - E_{tv}, & S_{tt} \in [S_{tt}^-, S_{tt}^+], & S_{tt}^- = \widehat{S}_{tt} - E_{\downarrow}, \\ S_{tv}^+ &= \widehat{S}_{tv} + E_{tv}, & S_{tt}^+ &= \widehat{S}_{tt} + E_{\downarrow} + E_{\uparrow}. \end{aligned}$$

So the true pair (S_{tv}, S_{tt}) lies somewhere in the rectangle $[S_{tv}^-, S_{tv}^+] \times [S_{tt}^-, S_{tt}^+]$, and the range of the ratio $\beta = S_{tv}/S_{tt}$ over that rectangle is exactly the enclosure of Theorem 1.

Proof of Theorem 1. Consider $f(p, q) = p/q$ on the rectangle; since $S_{tt}^- > 0$, every point has $q > 0$ and f is finite. Fix any $q > 0$: f is linear in p , so its extreme over $p \in [S_{tv}^-, S_{tv}^+]$ is attained at an endpoint, giving S_{tv}^-/q or S_{tv}^+/q . Each of S_{tv}^-/q and S_{tv}^+/q is monotone in q on $q > 0$ (its derivative, $-S_{tv}^-/q^2$ resp. $-S_{tv}^+/q^2$, keeps one sign), so its extreme over $q \in [S_{tt}^-, S_{tt}^+]$ is attained at $q = S_{tt}^-$ or $q = S_{tt}^+$. Hence the maximum and minimum of f over the whole rectangle lie among the four corner ratios $S_{tv}^{\pm}/S_{tt}^{\pm}$ —this needs only $q > 0$, not that p keep one sign. Since the true (S_{tv}, S_{tt}) is a point of the rectangle, its ratio β lies between the smallest and largest of those corner ratios, which is the interval $[\beta^-(\sigma), \beta^+(\sigma)]$ of (1). $\square$

The degenerate case. If $S_{tt}^- \leq 0$ (or the estimate $\widehat{S}_{tt} \leq 0$), the denominator interval $[S_{tt}^-, S_{tt}^+]$ straddles 0. Then β is genuinely unbounded—some point clouds consistent with the summaries make S_{tt} arbitrarily small and the slope blows up—so the only honest answer is $(-\infty, +\infty)$, and that is what the routine returns. It is a valid (if useless) enclosure.

3.2.8 What the bound is, in one paragraph

We could not compute the slope because the raw points were discarded and many point clouds fit the same summaries. We replaced the two unknown means by computable midpoint surrogates (Lemma A), split each of S_{tt}, S_{tv} into an exact between/within decomposition, and bounded every place an unknown entered using only two facts: each point's t sits within its bucket's width w_i of the centroid ($\star$), and each point's v sits within the stored $[\ell_i, h_i]$. That produced a computable estimate and a computable error bar for the numerator ($\hat{S}_{tv} \pm E_{tv}$) and for the denominator ($\hat{S}_{tt}$, with $-E_{\downarrow} / +E_{\downarrow} + E_{\uparrow}$). The true (S_{tv}, S_{tt}) lives in the resulting rectangle, and a ratio is monotone on a positive rectangle, so the four corner ratios bracket the true slope. Every error term is proportional to a bucket *width*: shrink the buckets and the interval shrinks with them. That is the whole proof, and the practical lever.

Numerical note. Everything above is exact in real arithmetic. A floating-point implementation should widen the final interval by the accumulated round-off (roughly machine epsilon times the magnitude of the sums). That is slack, not a gap in the argument.

Table 3: MNF pattern search: the AQP baseline versus VASTA and the exact cache, over the 24-query workload (ground truth OLS-0).

Method	Precision	Recall	Time/query	Peak mem
OLS-C (exact cache)	1.00	1.00	8.6 s	0.3 MB
VASTA	0.97	0.96	2.4 s	1.6 MB
AQP (sampled + exact fallback)	0.75	0.55	97 s	8.0 MB

4 Experiments

For the reviewers: This part contains new experiments as requested by the reviews.

4.1 Comparison against an AQP baseline

Section II-D of the paper argues that approximate query processing (AQP) is *complementary* to VASTA rather than a competing baseline: AQP approximates an aggregate value, whereas VASTA needs the four aggregates exactly to keep its pixel-error and slope-enclosure bounds deterministic. Here we back that argument with measurement.

The baseline. We had in fact considered such an AQP-style approach early in VASTA’s design, so a working implementation was already at hand and these results were readily obtained. We build the fairest possible AQP competitor and implement it inside VASTA’s own codebase, running in the same experimental harness so the comparison changes only the slope estimator and nothing else: a sampling estimator that runs the *same* pipeline as VASTA—the same candidate segments, the same angle bands, the same ε -NFA matcher, the same tolerance ρ_P , the same refine-until-tolerance loop, and the *same* exact strict-OLS fallback—with one substitution. Where VASTA derives a segment’s slope *interval* deterministically from the four cached aggregates, the baseline estimates the slope from a uniform random sample of the segment’s raw points and attaches a confidence interval to it, closed-form or bootstrap, in the manner of sampling-based AQP systems [1, 2]. Sampling is done in the database, stratified by calendar bucket, so refinement draws more points only where it needs them.

Setup. Ground truth is the exact no-cache OLS matcher (OLS-0). We pair a returned match with a ground-truth match when both endpoints agree to within one time unit (the paper’s tolerance), and report per-query precision, recall, mean query time, and peak memory (the cache footprint the harness records) on the MNF pattern-query workload (24 queries).

What goes wrong, and why. The AQP baseline loses on every axis. Each failure has one plain cause.

1. **It is far slower—about $40\times$ Vasta and $11\times$ the exact cache.** To place a segment inside or outside an angle band, the estimator’s interval must be narrow. That interval is $\hat{\beta} \pm z \cdot \text{se}$ about the sampled slope, where $\text{se} = \sqrt{\text{SSE}/((n-2)S_{tt})}$ is the textbook OLS standard error over the n sampled points—the closed form the baseline computes, and the normal-theory interval of sampling-based AQP [1, 2]. As the sample grows, S_{tt} increases in proportion to n while the residual variance $\text{SSE}/(n-2)$ settles to a constant, so the half-width $z \cdot \text{se}$ shrinks only as $1/\sqrt{n}$: halving it costs four times the points. When a segment’s true slope sits near a band edge this

drives the sample toward the whole segment, and when it still cannot decide the run reads the exact data, paying once for the sampling and again for that exact read. VASTA pays neither, for a concrete reason: its slope interval is a *deterministic enclosure* computed from the four cached aggregates (Section 3), never from sampled points, so it sharpens by reading the finer aggregate buckets already in the cache—which roll up exactly and are reused across queries—and its exact fallback is a bounded last resort over only the still-ambiguous regions, which this workload rarely reaches.

2. **It misses almost half the true matches (recall 0.55).** A sampled slope with a confidence interval is an *estimate*, not a bound. Nothing forces the true slope to lie in the reported interval, so that interval can fall wholly outside the angle band while the true slope is inside it, and the matcher rejects a genuine match. Once the region holding it has no candidates left in doubt, the exact fallback—which revisits only still-ambiguous regions—never looks there again, and the match is lost. VASTA cannot lose a match this way: by the slope-enclosure result of Section 3 its interval provably *contains* the true slope, so a genuine match’s interval always intersects the band and is never rejected; it stays in doubt until exact data confirms it, which is why VASTA’s recall stays near 1.0.
3. **One in four returned matches is spurious (precision 0.75).** Sampling makes the opposite mistake too: an estimate can land inside the band by chance even when the true slope is outside, so a non-match is confirmed. VASTA’s confident matches are provably inside the band, so it almost never does this.
4. **Its memory grows too—8.0 MB against Vasta’s 1.6 MB and the exact cache’s 0.3 MB.** To sharpen an interval the sampler must *materialize* the raw points it draws, and it draws the most exactly where segments are hardest, holding them for the duration of the query; on fallback it then materializes the exact data on top. VASTA holds only the compact (min, max, count, sum) buckets, whose footprint is set by the resolution rather than by how many raw points a segment contains.

It also cannot serve the view—so it can only be a silo. The point of VASTA is a *single* shared cache—the (min, max, count, sum) buckets—that answers both the render and the search, so work done for one is reused by the other. The sampling estimator has nothing the view can reuse: it produces a slope from a random handful of raw points, keeps no bucket cache, and a random sample cannot give the min/max envelope an error-bounded render needs. So an AQP pattern searcher cannot share state with the visualization side at all; it would have to sit beside a *separate* visualization stack—exactly the siloed, two-state design the shared cache removes. Beyond being slower and less accurate on the search alone, it forfeits the cross-operation reuse that is the whole contribution.

Either way, complementary not competing. Two honest readings both condemn AQP as a competitor. *As measured*, it drops half the true matches, returns a quarter spurious, and runs an order of magnitude slower. And *if* one insisted on pushing its recall up to VASTA’s, the only way is to stop discarding any candidate that ever touched a band and exact-verify them all—which is just OLS-C with wasted sampling on top. In neither reading does it compete; it is complementary, and it cannot even be posed on one shared cache the way VASTA is.

References

- [1] S. Agarwal, H. Milner, A. Kleiner, A. Talwalkar, M. I. Jordan, S. Madden, B. Mozafari, and I. Stoica. Knowing when you're wrong: Building fast and reliable approximate query processing systems. In *SIGMOD*, pages 481–492, 2014.
- [2] S. Agarwal, B. Mozafari, A. Panda, H. Milner, S. Madden, and I. Stoica. Blinkdb: Queries with bounded errors and bounded response times on very large data. In *EuroSys*, pages 29–42, 2013.
- [3] S. Maroulis, V. Stamatopoulos, G. Papastefanatos, and M. Terrovitis. Visualization-aware time series min-max caching with error bound guarantees. *Proc. VLDB Endow.*, 17(8):2091–2103, 2024.
- [4] T. Siddiqui, P. Luh, Z. Wang, K. Karahalios, and A. G. Parameswaran. Shapesearch: A flexible and efficient system for shape-based exploration of trendlines. In *SIGMOD*, pages 51–65, 2020.